

Success Evaluation Report — PR #7875

Tile Engine → Dispatcher GEMM Codegen Parity Bring-Up

Author / Reviewer: M. E. Ozturk · Branch: muozturk/dispatcher-te-parity · Date: 2026-06-02

1. Executive Summary

PR #7875 delivers the **parity bring-up suite** that proves the CK Tile *dispatcher* codegen path reproduces the established *Tile Engine* (TE) GEMM kernels. The work covers both Phase 1 (the "smallest possible bridge" — translator, identifier oracle, single-kernel harness) and a substantial slice of Phase 2 (sweep runner, comparison report, C-API scaffolding).

****Overall assessment:** a strong, well-engineered foundation that meets the spirit of Phase 1 and most of Phase 2 — with one structural caveat that bounds every numerical and performance claim.**

Dimension	Verdict
Phase 1 (T1.1–T1.7)	~70% complete — translator + identifier parity are production-grade; GPU stages verified on gfx942
Phase 2 (T2.1–T2.7)	~50% complete — sweep + report run; T2.2 .so round-trip unproven
Overall (weighted)	~55–60%
Headline risk	No Tile Engine build was ever available → "parity" is dispatcher self-consistency, not cross-stack equivalence

The code quality, documentation honesty, and test discipline are above bar. The gap is one of *evidence*, not *engineering*: the project's defining claim — dispatcher ≡ Tile Engine on numbers and throughput — has not yet been demonstrated end-to-end because no TE binary was accessible in the environment.

2. Objective & Scope

The task (per the project description) is to prove the new dispatcher codegen produces kernels that are **equivalent** to Tile Engine across four axes:

- Identifier parity** — the registry key matches byte-for-byte, offline and at runtime.
- Numerical parity** — $C = A \cdot B$ matches a trusted reference within tolerance.
- Performance parity** — throughput is within 2% of TE.
- Coverage** — across dtypes (fp16/bf16/fp8/int8), layouts, pipelines, and sizes.

Phase 1 was scoped as a *single hand-picked example* end-to-end; Phase 2 generalizes to a full sweep with reporting.

3. What Was Delivered

Phase 1 — The Bridge

- `te_to_dispatcher.py` — TE config JSON → canonical dispatcher `KernelKey` dicts.

Applies every TE→dispatcher mapping exactly once (scheduler default→auto, fp8/bf8 output→fp16, int8 acc→int32). Guards `split_k ∈ [1, 255]` (uint8_t overflow), rejects unsupported pipelines (compv1/compv2/preshufflev1) and the compv3+interwave trait combo with clear `TranslationErrors` instead of opaque codegen failures. **~95% — production-grade.**

- `identifier.py` + `cpp_identifier_oracle.cpp` + `check_identifier_parity.py` — two-oracle identifier check (Python vs the real C++ `KernelKey::encode_identifier()`), CPU-only, auto-recompiles on header change. **~85%.**

- `harness.cpp` — single-kernel GPU harness; dual tolerance gate ($\text{abs} = 1\text{e-}3 \cdot \sqrt{K}$, $\text{rel} = 1\text{e-}2$); fp8/bf8 verification skipped with timing retained. **~75%** (note: it links the kernel directly via `CK_TILE_SINGLE_KERNEL_INCLUDE`, bypassing the runtime registry lookup — see §6).

- `check_parity.py` — three-stage orchestration (identifier → numerical → perf), GPU stages gated, perf = median of 10 runs at 2% tolerance.

GPU-verified on gfx942 (MI300X), 2026-06-02: fp16/bf16/fp8/int8/split_k rcr
single-kernel configs passed identifier + numerical + timing stages at $512^3/1024^3/2048^3$.

Phase 2 — Generalization

- `sweep_runner.py` — drives codegen→build→harness per (kernel, problem) combo via subprocess; per-combo try/except; incremental resume; Parquet output. Works **without** the unbuilt C-API. **~80%.**
- `compare_report.py` — Markdown/HTML report with rollups by dtype/layout/pipeline/tile and an optional two-stack --te merge. **~75%** (TE columns empty — no baseline).
- `dispatcher_capi.{h,cpp}` + `dispatcher_binding.py` + `demo_binding.py` — multi-kernel C-API ctypes scaffolding. **~35% — written, not built.**
- `PORTING_DECISIONS.md` — the living design-decisions doc; corrected this PR to remove three overclaims (see §5). **~85%.**

4. Per-Task Scorecard

Task	Description	Score	Status
T1.1	TE→dispatcher translator	95%	Strong
T1.2	Identifier oracle parity	85%	Strong
T1.3	Codegen drive	80%	Good
T1.4	Single-kernel harness	75%	Good (registry bypass)

T1.5	Numerical verification	70%	Self-consistency only
T1.6	3-stage orchestration	75%	Good
T1.7	Performance gate	45%	No TE baseline
T2.1	Config sweep expansion	70%	Good
T2.2	Multi-kernel C-API .so	35%	Unbuilt
T2.3	Sweep runner	80%	Strong
T2.4	Full CI sweep	20%	Not run
T2.5	Cross-stack comparison	25%	No TE data
T2.6	Comparison report	75%	Good (TE cols empty)
T2.7	Decisions documentation	85%	Strong

5. Engineering Quality Highlights

- **Honest documentation.** This PR corrected `PORTING_DECISIONS.md` where it had claimed a `double_buffer` discrepancy between translator and codegen. Investigation showed codegen sets ``DoubleSmemBuffer = (pipeline == "compv4" or pipeline == "preshufflev2")`` — i.e. the two **agree**. The doc, the follow-up issue, and a self-contradicting unit test were all corrected to reflect reality.
- **Failure-first defaults.** Unsupported pipelines and out-of-range `split_k` are rejected at translation time with actionable messages, not deep in codegen.
- **Test discipline.** 60 translator unit tests pass; the test suite explicitly covers the four T1.1 done-criteria (vanilla fp16 rcr, padding, `split_k`, persistent).
- **Crash-safe sweeps.** Incremental resume and per-combo isolation make multi-hour sweeps restartable.

6. The Structural Limitation

No Tile Engine build was available in the environment. This single fact bounds the project's central claim:

- Numerical stages (T1.5, T2.5) verify each stack against **its own** CPU fp32 reference. Because the dispatcher and TE harnesses initialize A/B with **different** input data, this proves *self-consistency of the dispatcher kernel* — not that dispatcher and TE produce *identical* C matrices.
- Performance stages (T1.7, T2.6) report **dispatcher-only** throughput. The 2% gate is implemented and exercised, but there is no TE number to gate against.
- The harness links the kernel directly (`CK_TILE_SINGLE_KERNEL_INCLUDE`) rather than going through the dispatcher registry lookup, so the *runtime* dispatch path is identifier-verified but not numerically exercised end-to-end.

None of this is a code defect — it is missing evidence. Closing it requires (a) a TE

build via `--te-build-dir`, and (b) building/exercising the T2.2 .so round-trip.

7. Risks & Follow-Ups

#	Item	Priority
1	Obtain a TE build; run true cross-stack numerical + perf comparison	High
2	Build the T2.2 .so; demonstrate Python→registry→GEMM→verify round-trip	High
3	Exercise the runtime registry path in the numerical harness (not just identifier)	Medium
4	Run the full CI sweep (T2.4) and publish a populated comparison report	Medium
5	Generalize harness strides beyond rcr	Low

8. Verdict

PR #7875 is a **successful Phase 1 bring-up and a credible Phase 2 scaffold**. The translator and identifier-parity machinery are the strongest parts and are effectively production-ready. The documentation is unusually honest about its own gaps, which is exactly what a parity effort needs.

The work should be regarded as *****parity infrastructure proven on the dispatcher side, cross-stack equivalence pending a Tile Engine build.** With a TE binary and the C-API round-trip, the remaining ~40% closes quickly because the surrounding harness, sweep, and reporting are already in place.

****Recommendation:** merge Phase 1 as a foundation; track the TE-build comparison and the T2.2 round-trip as the two gating follow-ups before declaring full parity.**

Generated for internal MLSE review.